



Digi-ID

Digi-ID.io
DigiByte.org
DigiByte.io

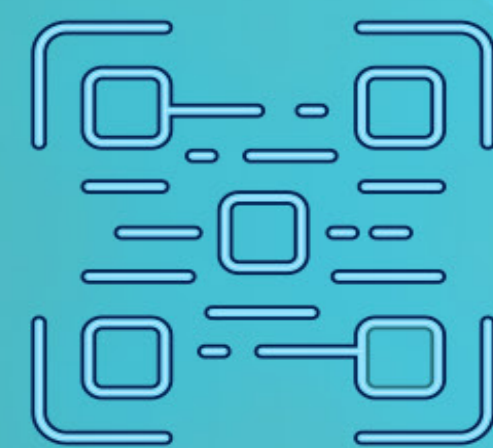


Digi-ID: Integration Guide

**A practical guide for website, service, and
wallet developers**

Authored by the DigiByte Community.

Latest revision:
September 2026



Digi-ID Overview



What is Digi-ID?

Digi-ID is a free, open-source authentication protocol. It lets users sign in to supported websites and services by scanning or tapping a QR code and approving the request in a compatible app. The app sends a digital signature instead of a reusable password.

A service can use Digi-ID instead of a password or require it as an additional authentication step. Offering two sign-in choices does not automatically add a second security check.

How it works:

- **Generate:** The service creates a unique sign-in request and displays it as a QR code.
- **Scan:** The user scans or taps the code with a compatible Digi-ID app or wallet.
- **Approve:** The user approves the request, and the app signs the complete Digi-ID URI.
- **Verify:** The service validates the response and grants access to the associated account or permissions.

The app signs the request locally. Digi-ID does not send the private key to the service during sign-in. The service uses the verified Digi-ID address to identify the account and keeps the challenge and session information needed to validate each request.

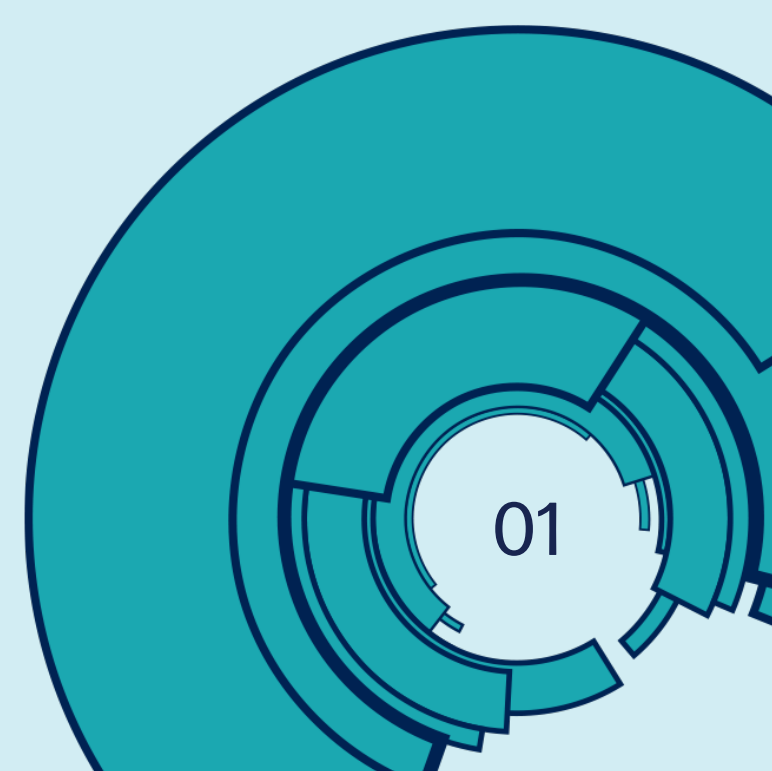
Why use it?

- **Fewer password-related risks:** When Digi-ID replaces password sign-in, users do not need a website password that could be guessed, stolen, or reused. Devices, sessions, and account recovery still need protection.
- **Minimal data exposure:** Authentication does not inherently transmit a name, date of birth, phone number, or cryptocurrency balance.
- **Separate login addresses:** Digi-ID derives addresses from the callback URL. This helps avoid using one login address across unrelated services.
- **Simple sign-in:** Users scan or tap a QR code and approve the request in their app.
- **Flexible deployment:** It can support website login, applications, gaming accounts, shopping, access control, and connected devices.
- **Open and free:** Digi-ID is an open-source, community-driven protocol offered free to use. Development, hosting, and support may still involve costs.

For service providers:

The QR code contains a Digi-ID URI with a callback URL and a “nonce,” a unique value for that authentication request. The server validates the returned URI, nonce, signature, and Digi-ID address before granting access to the associated account or permissions.

PHP and JavaScript examples are available, and signature verification does not require a DigiByte node. Use HTTPS in production, expire challenges, and reject challenges that have already been used. Digi-ID does not replace secure session handling or other application security controls.

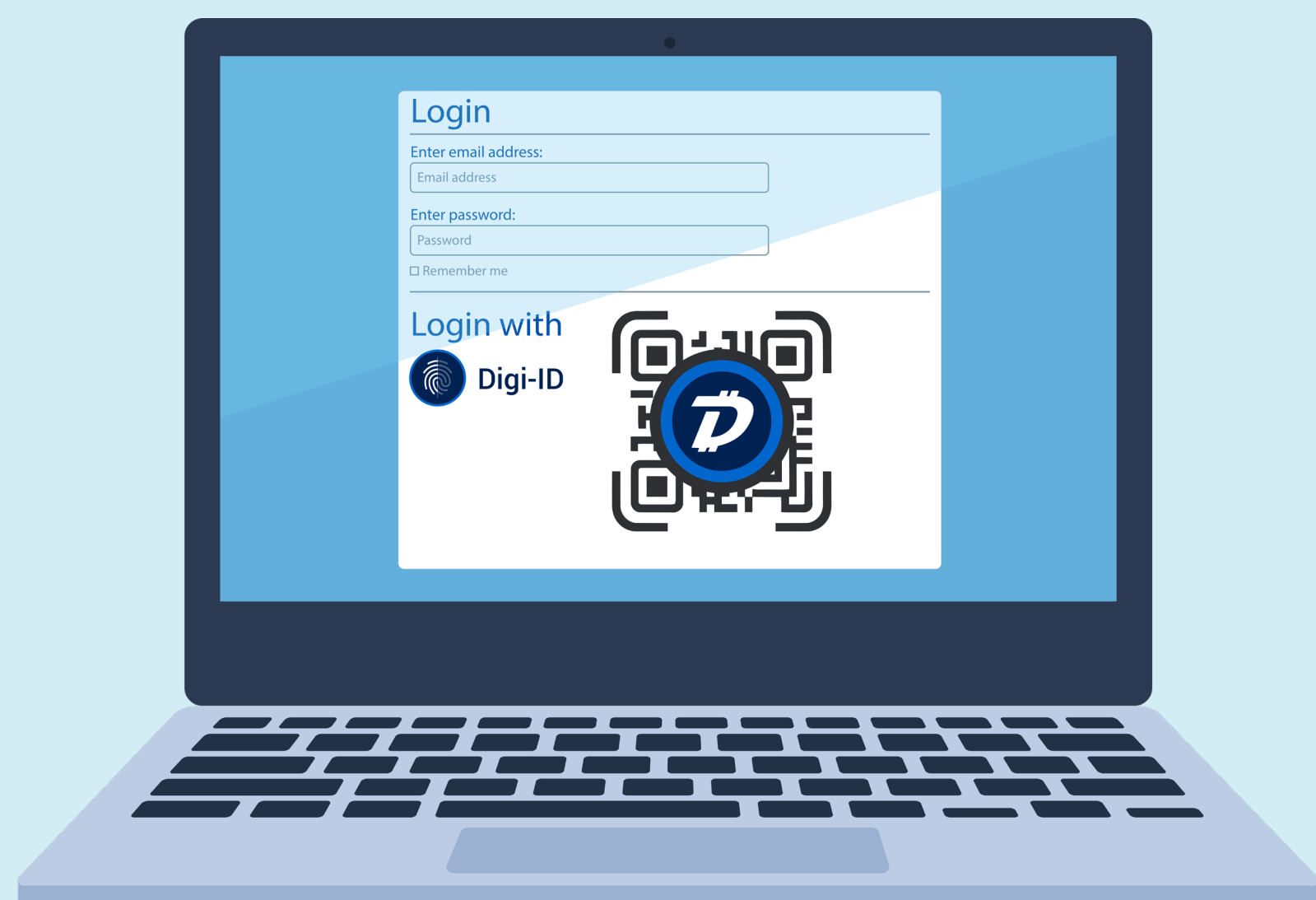


Website Integration



To add Digi-ID sign-in, your service creates an authentication request, displays it as a QR code, and validates the app's response. The following steps explain how the website and app work together.

Before accessing a secure or restricted area of a website or service, the user will see a QR code, potentially alongside existing authentication methods:



Sign in flow:

1. Your service creates a unique authentication request, links it to a pending session, and displays a Digi-ID QR code.
2. The user scans or taps the code with a compatible Digi-ID app or wallet and approves the request.
3. The app signs the complete Digi-ID URI and sends the response to your callback URL.
4. Your server validates the response using the checks under “Manage the authentication session” (page 03), then grants access to the associated account.

Create the QR Code:

- The QR code contains the Digi-ID authentication URI; the app sends its response to an HTTP(S) callback URL. The link follows this format:
`digiid://www.website.com/callback?x=NONCE`
- `digiid` identifies the protocol and opens a compatible mobile app when the link is tapped.
- `www.website.com/callback` identifies your callback endpoint, the URL where your server receives the authentication response.
- `x=NONCE` contains a unique value linked to the user's session ID. A nonce is a value used for one authentication request.
- Use a plain callback path with no query string of its own. The sign-in request carries the nonce as `?x=NONCE`, and the
- JavaScript library expects the callback URL to have no parameters of its own.

Note: For HTTP testing, append `&u=1` to the Digi-ID URI encoded in the QR code. This tells the app to use HTTP for the callback. Use HTTPS in production. Expire each authentication request after a short window. Around 90 seconds is a reasonable starting point for website sign-in; shorten or lengthen it to suit your application.

Label the sign-in option:

Place “Log in with Digi-ID” above or beside the QR code so users know what it does. The DigiByte “D” logo inside the code is optional.

Key resources:

[PHP implementation](#)

[JavaScript library](#)

The PHP example also requires a MySQL database and the GMP extension for PHP. It relies on a fixed, dated copy of an external elliptic-curve library, which should be reviewed and updated before production use. Both library examples require compatibility and security review.

Server Setup & Other Uses



Manage the authentication session

- Generate a fresh, unpredictable nonce for each authentication request and link it to the pending session.
- Store the request's expiration time and whether it has already been used.
- Check the returned URI against the expected callback and challenge.
- Verify the signature and the returned Digi-ID address.
- Reject unknown, expired, or previously used challenges. Mark an accepted challenge as used so concurrent responses cannot accept it twice.
- Grant access only to the account or permissions associated with the verified address, then establish the authenticated session securely.

Use Digi-ID for building access

1. Place a screen near each door or restricted area and display a Digi-ID QR code.
2. Use a single callback URL for the whole system. Generate a fresh nonce for each door display and record which door that nonce belongs to, so the response identifies the door.
3. The user scans the code with a compatible Digi-ID app or wallet and confirms the request.
4. Your central authentication server receives the response, looks up the door from the nonce, and checks whether the user's verified Digi-ID address has access to that door.
5. If the user is authorized, the server unlocks the door. Require a fresh Digi-ID approval before sensitive actions such as changing account details, adding a device, or making a payment, and tell users to approve only codes shown on a page they opened themselves.

Note: Do not give each door its own callback URL. The derived Digi-ID address depends on the callback URI, including its path (see page 04), so per-door callback URLs produce a different address for the same user at every door and a single enrollment will not carry across them.

Store the user's verified Digi-ID address as an account identifier. Keep the challenge and session information needed to validate each request. The server does not need the user's private key.

Do I need a DigiByte node?

No. Your server can verify Digi-ID signatures without a DigiByte node, blockchain connection, or external identity provider. Your application still needs its normal server, storage, and session handling.

Can Digi-ID work without public internet access?

Yes, if the app can reach your callback URL over a local or private network. A public internet connection is not required, but the app must be able to communicate with your server.



Wallet Integration



This section explains how to derive a Digi-ID authentication key and check the result against a test vector.

How does the callback URI affect the address?

In the derivation procedure shown in this guide, the key depends on the wallet seed, index, and callback URI, including its path. Changing the callback URI can change the derived address, even when the domain stays the same.

Derive the authentication key

An HD (hierarchical deterministic) wallet uses a single seed to generate its keys.

Definitions:

- **Callback URI:** the HTTP or HTTPS callback address used for key derivation. For the procedure and test vector shown here, include the scheme, host, and path. Do not use the `digiid://` authentication URI or include its nonce parameter.
- **index:** an unsigned 32-bit integer used to derive additional keys for the same callback URI. Use 0 if no index is provided.

Process:

1. Encode index as four bytes in little-endian order, with the least significant byte first.
2. Append the UTF-8 bytes of the callback URI.
3. Calculate the SHA-256 hash of the combined bytes.
4. Take the first 16 bytes of the hash and read them as four unsigned 32-bit little-endian integers.
5. Clear the highest bit of each integer using value `& 0x7fffffff`. Call the resulting values A, B, C, and D.
6. Derive the BIP32 path `m/13'/A'/B'/C'/D'`. Each apostrophe applies hardened derivation once.

Test values from the source guide

These values have been verified end to end: the callback URI and index reproduce the hash result, the hash reproduces the BIP32 path, and the path reproduces the Digi-ID address shown.

Use this published mnemonic for testing only. Do not use it for a real wallet.

- **BIP39 mnemonic:**
myth glimpse mystery abstract embark net faint hospital catch hint develop state
BIP39 passphrase: empty string ("").
- **Authentication URI:** `digiid://digiid.digibyteprojects.com/callback?x=c6140375e5bae71e`
- **Callback URI:** the HTTP or HTTPS callback URL used for key derivation.
- **Index:** 0
- **Hash result:** `460ebbfd1df7410106d62d41f802f769bc00db825e9cf8c649cb069dec35e8fb`
- **BIP32 path:** `m/13'/2109410886'/21100317'/1093522950'/1777795832'`
- **Digi-ID address:** `DJDAkjie6nrW6RpFZSTpNUXsZ9JE2x6p1o`



Digi-ID

Digi-ID.io
DigiByte.org
DigiByte.io



Questions? Contact Us.

Development Support:

Developer channels exist on Telegram, Discord, and Gitter. GitHub Discussions on the DigiByte Core repository is another route for development questions:

https://app.gitter.im/#/room/#DigiByte-Core_community:gitter.im

Updates / New Releases:

We recommend subscribing to the GitHub repository to keep informed about all updates, releases, and any information pertaining to DigiByte development.

Watch the Digi-ID reference repositories for changes:

<https://github.com/DigiByte-Core/digiid-php>

<https://github.com/DigiByte-Core/digiid-js>

Both are stable but dormant, so also track your own dependency chain. For DigiByte Core node releases, which are separate from Digi-ID, see:

<https://github.com/DigiByte-Core/digibyte/releases>

General Questions:

If you have any questions about this documentation or wish to communicate with DigiByte community support, reach out to [@DigiAwareness_](#) with requests.